

Tutorial Completo: Criando e Enviando Dados para APIs com JavaScript

1. Métodos de API (HTTP Methods)

APIs REST utilizam métodos HTTP para realizar diferentes operações:

- **GET**: Busca dados (ler)
- **POST**: Cria novos dados
- **PUT**: Atualiza dados completamente
- **PATCH**: Atualiza dados parcialmente
- **DELETE**: Remove dados

Pense nisso como ações em um caderno:

- GET = ler uma página
- POST = escrever uma nova página
- PUT = reescrever uma página inteira
- PATCH = corrigir algo em uma página
- DELETE = arrancar uma página

2. Métodos POST, PUT e DELETE

POST - Criar Novos Dados

O método POST envia dados para criar um novo recurso no servidor.

```
javascript

// Estrutura básica de uma requisição POST
fetch('https://api.exemplo.com/usuarios', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    nome: 'Maria Silva',
    email: 'maria@email.com'
  })
});
```

Componentes importantes:

- `method: 'POST'` - define o tipo de operação
- `headers` - informa o formato dos dados (JSON)
- `body` - os dados que você está enviando (convertidos para texto com `JSON.stringify()`)

PUT - Atualizar Dados Completamente

O PUT substitui completamente um recurso existente.

```
javascript

// Atualizar usuário com ID 5
fetch('https://api.exemplo.com/usuarios/5', {
  method: 'PUT',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    nome: 'Maria Silva Santos',
    email: 'maria.santos@email.com',
    telefone: '11999999999'
  })
});
```

Diferença entre PUT e PATCH:

- PUT: envia TODOS os campos (substitui tudo)
- PATCH: envia apenas os campos que mudaram

DELETE - Remover Dados

O DELETE remove um recurso do servidor.

```
javascript

// Deletar usuário com ID 5
fetch('https://api.exemplo.com/usuarios/5', {
  method: 'DELETE'
});
```

Observe que o DELETE geralmente não precisa de `body`, apenas a URL com o ID do recurso.

3. Criação de Objetos e Envio via Fetch

Passo a Passo

1. Criar o objeto JavaScript:

javascript

```
const novoProduto = {  
  nome: 'Notebook',  
  preco: 2500,  
  estoque: 10  
};
```

2. Converter para JSON:

javascript

```
const produtoJSON = JSON.stringify(novoProduto);  
// Resultado: '{"nome":"Notebook","preco":2500,"estoque":10}'
```

3. Enviar com fetch:

javascript

```
fetch('https://api.exemplo.com/produtos', {  
  method: 'POST',  
  headers: {  
    'Content-Type': 'application/json'  
  },  
  body: produtoJSON  
)  
  .then(response => response.json())  
  .then(data => console.log('Sucesso:', data))  
  .catch(error => console.error('Erro:', error));
```

Exemplo Completo

javascript

```
async function criarProduto() {  
  const produto = {  
    nome: 'Mouse Gamer',  
    preco: 150,  
    categoria: 'Periféricos'  
  };  
  
  try {  
    const response = await fetch('https://api.exemplo.com/produtos', {  
      method: 'POST',  
      headers: {  
        'Content-Type': 'application/json'  
      },  
      body: JSON.stringify(produto)  
    });  
  
    if (!response.ok) {  
      throw new Error('Erro na requisição');  
    }  
  
    const resultado = await response.json();  
    console.log('Produto criado:', resultado);  
  } catch (erro) {  
    console.error('Falha ao criar produto:', erro);  
  }  
}
```

4. Exercícios Práticos com POST, PUT e DELETE

Exercício 1: POST - Criar um Usuário

javascript

```
async function exercicioPOST() {  
  const novoUsuario = {  
    nome: 'João Santos',  
    email: 'joao@email.com',  
    idade: 25  
  };  
  
  try {  
    const response = await fetch('https://jsonplaceholder.typicode.com/users', {  
      method: 'POST',  
      headers: {  
        'Content-Type': 'application/json'  
      },  
      body: JSON.stringify(novoUsuario)  
    });  
  
    const resultado = await response.json();  
    console.log('Usuário criado:', resultado);  
  } catch (erro) {  
    console.error('Erro:', erro);  
  }  
}  
  
// Chame a função para testar  
exercicioPOST();
```

Exercício 2: PUT - Atualizar um Post

javascript

```
async function exercicioPUT() {
  const postAtualizado = {
    id: 1,
    title: 'Título Atualizado',
    body: 'Conteúdo completamente novo',
    userId: 1
  };

  try {
    const response = await fetch('https://jsonplaceholder.typicode.com/posts/1', {
      method: 'PUT',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(postAtualizado)
    });

    const resultado = await response.json();
    console.log('Post atualizado:', resultado);
  } catch (erro) {
    console.error('Erro:', erro);
  }
}

exercicioPUT();
```

Exercício 3: DELETE - Remover um Item

javascript

```
async function exercicioDELETE() {
  try {
    const response = await fetch('https://jsonplaceholder.typicode.com/posts/1', {
      method: 'DELETE'
    });

    if (response.ok) {
      console.log('Item deletado com sucesso!');
    }
  } catch (erro) {
    console.error('Erro ao deletar:', erro);
  }
}

exercicioDELETE();
```

Exercício 4: Desafio Completo

Crie uma função que cadastre um livro, depois atualize seu preço e, por fim, o delete:

javascript

```
async function desafioCompleto() {
  const API_URL = 'https://jsonplaceholder.typicode.com/posts';

  // 1. Criar livro (usando posts como exemplo)
  const novoLivro = {
    title: 'JavaScript Avançado',
    body: 'Um guia completo sobre JS',
    userId: 1
  };

  try {
    // POST
    const respostaCriacao = await fetch(API_URL, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(novoLivro)
    });
    const livroCriado = await respostaCriacao.json();
    console.log('1. Livro criado:', livroCriado);

    // PUT - Atualizar
    const livroAtualizado = {
      ...livroCriado,
      title: 'JavaScript Avançado - 2ª Edição'
    };

    const respostaAtualizacao = await fetch(`${API_URL}/${livroCriado.id}`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(livroAtualizado)
    });
    const livroAtual = await respostaAtualizacao.json();
    console.log('2. Livro atualizado:', livroAtual);

    // DELETE
    const respostaDeletar = await fetch(`${API_URL}/${livroCriado.id}`, {
      method: 'DELETE'
    });

    if (respostaDeletar.ok) {
      console.log('3. Livro deletado com sucesso!');
    }

  } catch (erro) {
    console.error('Erro no processo:', erro);
  }
}
```

```
}  
  
desafioCompleto();
```

5. Manipulação de Resposta da API

Tipos de Resposta

Quando você faz uma requisição, a API retorna uma **resposta (response)** que contém:

1. **Status Code** (código de status):
 - 200-299: Sucesso
 - 400-499: Erro do cliente (dados inválidos, não autorizado)
 - 500-599: Erro do servidor
2. **Headers** (cabeçalhos): informações sobre a resposta
3. **Body** (corpo): os dados propriamente ditos

Verificando o Status

```
javascript  
  
async function verificarStatus() {  
  const response = await fetch('https://api.exemplo.com/dados');  
  
  console.log('Status:', response.status); // Ex: 200  
  console.log('OK?', response.ok); // true se status entre 200-299  
  
  if (!response.ok) {  
    throw new Error(`Erro HTTP! Status: ${response.status}`);  
  }  
  
  const dados = await response.json();  
  return dados;  
}
```

Convertendo a Resposta

```
javascript
```

// Para JSON (mais comum)

```
const dados = await response.json();
```

// Para texto simples

```
const texto = await response.text();
```

// Para Blob (arquivos, imagens)

```
const blob = await response.blob();
```

Acessando os Dados

javascript

```
async function acessarDados() {  
  const response = await fetch('https://jsonplaceholder.typicode.com/users/1');  
  const usuario = await response.json();  
  
  // Acessar propriedades do objeto  
  console.log('Nome:', usuario.name);  
  console.log('Email:', usuario.email);  
  console.log('Cidade:', usuario.address.city);  
}
```

Tratamento de Erros Completo

javascript

```
async function requisicaoCompleta() {
  try {
    const response = await fetch('https://api.exemplo.com/dados');

    // Verifica se a resposta foi bem-sucedida
    if (!response.ok) {
      throw new Error(`Erro ${response.status}: ${response.statusText}`);
    }

    const dados = await response.json();
    return dados;

  } catch (erro) {
    // Trata erros de rede ou de conversão JSON
    if (erro instanceof TypeError) {
      console.error('Erro de rede:', erro.message);
    } else {
      console.error('Erro:', erro.message);
    }
    throw erro; // Repassa o erro se necessário
  }
}
```

6. Exercício Prático de Manipulação de Respostas

Exercício 1: Extrair e Exibir Dados

javascript

```
async function exercicioResposta1() {
  try {
    const response = await fetch('https://jsonplaceholder.typicode.com/users');

    // Verifica status
    if (!response.ok) {
      throw new Error('Falha ao buscar usuários');
    }

    const usuarios = await response.json();

    // Manipular os dados recebidos
    usuarios.forEach(usuario => {
      console.log(`${usuario.name} - ${usuario.email}`);
    });

    // Filtrar usuários de uma cidade específica
    const usuariosDeLondres = usuarios.filter(u =>
      u.address.city === 'Gwenborough'
    );

    console.log('Usuários encontrados:', usuariosDeLondres.length);

  } catch (erro) {
    console.error('Erro ao processar usuários:', erro);
  }
}

exercicioResposta1();
```

Exercício 2: Validar e Processar Resposta de POST

javascript

```
async function exercicioResposta2() {
  const novoPost = {
    title: 'Meu Post',
    body: 'Conteúdo do post',
    userId: 1
  };

  try {
    const response = await fetch('https://jsonplaceholder.typicode.com/posts', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(novoPost)
    });

    // Verificar status específico para POST (geralmente 201)
    console.log('Status da criação:', response.status);

    const postCriado = await response.json();

    // Validar se o post foi criado corretamente
    if (postCriado.id) {
      console.log('✓ Post criado com sucesso!');
      console.log('ID do novo post:', postCriado.id);
      console.log('Dados completos:', postCriado);
    } else {
      console.warn('⚠ Post criado mas sem ID');
    }

  } catch (erro) {
    console.error('✗ Falha ao criar post:', erro);
  }
}

exercicioResposta2();
```

Exercício 3: Combinar Múltiplas Respostas

javascript

```
async function exercicioResposta3() {
  try {
    // Buscar um usuário e seus posts simultaneamente
    const [respostaUsuario, respostaPosts] = await Promise.all([
      fetch('https://jsonplaceholder.typicode.com/users/1'),
      fetch('https://jsonplaceholder.typicode.com/users/1/posts')
    ]);

    const usuario = await respostaUsuario.json();
    const posts = await respostaPosts.json();

    // Combinar informações
    const relatorio = {
      nome: usuario.name,
      email: usuario.email,
      totalPosts: posts.length,
      titulosPosts: posts.map(p => p.title)
    };

    console.log('Relatório do Usuário:', relatorio);

  } catch (erro) {
    console.error('Erro ao gerar relatório:', erro);
  }
}

exercicioResposta3();
```

7. Simulação com JSON Server e MockAPI

Opção 1: JSON Server (Local)

Instalação:

```
bash

npm install -g json-server
```

Criar arquivo **db.json**:

```
json
```

```
{
  "produtos": [
    {
      "id": 1,
      "nome": "Notebook",
      "preco": 2500
    },
    {
      "id": 2,
      "nome": "Mouse",
      "preco": 50
    }
  ],
  "usuarios": [
    {
      "id": 1,
      "nome": "Ana",
      "email": "ana@email.com"
    }
  ]
}
```

Iniciar servidor:

```
bash
```

```
json-server --watch db.json --port 3000
```

Usar na aplicação:

```
javascript
```

// API rodando em <http://localhost:3000>

```
async function exemploJSONServer() {  
  // GET - Buscar todos os produtos  
  const response = await fetch('http://localhost:3000/produtos');  
  const produtos = await response.json();  
  console.log(produtos);  
  
  // POST - Criar novo produto  
  const novoProduto = {  
    nome: 'Teclado',  
    preco: 150  
  };  
  
  const respostaCriacao = await fetch('http://localhost:3000/produtos', {  
    method: 'POST',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify(novoProduto)  
  });  
  
  const produtoCriado = await respostaCriacao.json();  
  console.log('Produto criado:', produtoCriado);  
  
  // PUT - Atualizar produto  
  const respostaAtualizacao = await fetch(`http://localhost:3000/produtos/${produtoCriado.id}`, {  
    method: 'PUT',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify({ ...produtoCriado, preco: 180 })  
  });  
  
  // DELETE - Deletar produto  
  await fetch(`http://localhost:3000/produtos/${produtoCriado.id}`, {  
    method: 'DELETE'  
  });  
}
```

Opção 2: MockAPI (Online)

1. Acesse: <https://mockapi.io>
2. Crie uma conta gratuita
3. Crie um projeto e defina seus recursos

Exemplo de uso com MockAPI:

javascript

```
// Substitua YOUR_PROJECT_ID pelo ID do seu projeto
const API_BASE = 'https://YOUR_PROJECT_ID.mockapi.io/api/v1';

async function exemploMockAPI() {
  // Criar usuário
  const novoUsuario = {
    nome: 'Carlos',
    email: 'carlos@email.com',
    idade: 30
  };

  const response = await fetch(`${API_BASE}/usuarios`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(novoUsuario)
  });

  const usuario = await response.json();
  console.log('Usuário criado no MockAPI:', usuario);
}
```

Comparação das Ferramentas

Característica	JSON Server	MockAPI
Onde roda	Local (seu computador)	Nuvem (online)
Instalação	Requer Node.js	Sem instalação
Gratuito	Sim, 100%	Sim, com limites
Velocidade	Muito rápido	Depende da internet
Melhor para	Desenvolvimento local	Testes e demos

8. Exemplo de Atualização do DOM com Dados Criados

HTML Base

```
html
```

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CRUD de Produtos</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      max-width: 800px;
      margin: 50px auto;
      padding: 20px;
    }
    .produto-card {
      border: 1px solid #ddd;
      padding: 15px;
      margin: 10px 0;
      border-radius: 5px;
      background-color: #f9f9f9;
    }
    button {
      margin: 5px;
      padding: 8px 15px;
      cursor: pointer;
    }
    .btn-deletar { background-color: #ff4444; color: white; }
    .btn-editar { background-color: #4CAF50; color: white; }
    input, select {
      padding: 8px;
      margin: 5px;
      width: 200px;
    }
  </style>
</head>
<body>
  <h1>Gerenciador de Produtos</h1>

  <div id="formulario">
    <h2>Adicionar Produto</h2>
    <input type="text" id="inputNome" placeholder="Nome do produto">
    <input type="number" id="inputPreco" placeholder="Preço">
    <input type="number" id="inputEstoque" placeholder="Estoque">
    <button onclick="adicionarProduto()">Adicionar</button>
  </div>

  <div id="listaProdutos"></div>
```

```
<script src="app.js"></script>  
</body>  
</html>
```

JavaScript - Atualização do DOM

javascript

```
// app.js
```

```
const API_URL = 'http://localhost:3000/produtos';
```

```
// Função para buscar e exibir todos os produtos
```

```
async function carregarProdutos() {  
  try {  
    const response = await fetch(API_URL);  
    const produtos = await response.json();  
  
    const listaProdutos = document.getElementById('listaProdutos');  
    listaProdutos.innerHTML = ''; // Limpa a lista  
  
    produtos.forEach(produto => {  
      const card = criarCardProduto(produto);  
      listaProdutos.appendChild(card);  
    });  
  
  } catch (erro) {  
    console.error('Erro ao carregar produtos:', erro);  
    alert('Falha ao carregar produtos');  
  }  
}
```

```
// Função para criar o elemento visual do produto
```

```
function criarCardProduto(produto) {  
  const card = document.createElement('div');  
  card.className = 'produto-card';  
  card.innerHTML = `  
    <h3>${produto.nome}</h3>  
    <p><strong>Preço:</strong> R$ ${produto.preco}</p>  
    <p><strong>Estoque:</strong> ${produto.estoque} unidades</p>  
    <button class="btn-editar" onclick="editarProduto(${produto.id})">Editar</button>  
    <button class="btn-deletar" onclick="deletarProduto(${produto.id})">Deletar</button>  
  `;  
  return card;  
}
```

```
// Função para adicionar produto
```

```
async function adicionarProduto() {  
  const nome = document.getElementById('inputNome').value;  
  const preco = document.getElementById('inputPreco').value;  
  const estoque = document.getElementById('inputEstoque').value;  
  
  if (!nome || !preco || !estoque) {  
    alert('Preencha todos os campos!');  
    return;  
  }  
}
```

```
}

const novoProduto = {
  nome: nome,
  preco: parseFloat(preco),
  estoque: parseInt(estoque)
};

try {
  const response = await fetch(API_URL, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(novoProduto)
  });

  const produtoCriado = await response.json();

  // Adiciona o novo produto ao DOM sem recarregar tudo
  const listaProdutos = document.getElementById('listaProdutos');
  const card = criarCardProduto(produtoCriado);
  listaProdutos.appendChild(card);

  // Limpa o formulário
  document.getElementById('inputNome').value = '';
  document.getElementById('inputPreco').value = '';
  document.getElementById('inputEstoque').value = '';

  alert('Produto adicionado com sucesso!');

} catch (erro) {
  console.error('Erro ao adicionar produto:', erro);
  alert('Falha ao adicionar produto');
}

// Função para deletar produto
async function deletarProduto(id) {
  if (!confirm('Deseja realmente deletar este produto?')) {
    return;
  }

  try {
    await fetch(`${API_URL}/${id}`, {
      method: 'DELETE'
    });

    // Remove o card do DOM
  }
}
```

```
carregarProdutos(); // Recarrega a lista

alert('Produto deletado com sucesso!');

} catch (erro) {
  console.error('Erro ao deletar produto:', erro);
  alert('Falha ao deletar produto');
}
}

// Função para editar produto (simplificada)
async function editarProduto(id) {
  const novoPreco = prompt('Digite o novo preço:');

  if (!novoPreco) return;

  try {
    // Busca o produto atual
    const response = await fetch(`${API_URL}/${id}`);
    const produto = await response.json();

    // Atualiza apenas o preço
    const produtoAtualizado = {
      ...produto,
      preco: parseFloat(novoPreco)
    };

    await fetch(`${API_URL}/${id}`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(produtoAtualizado)
    });

    // Recarrega a lista
    carregarProdutos();

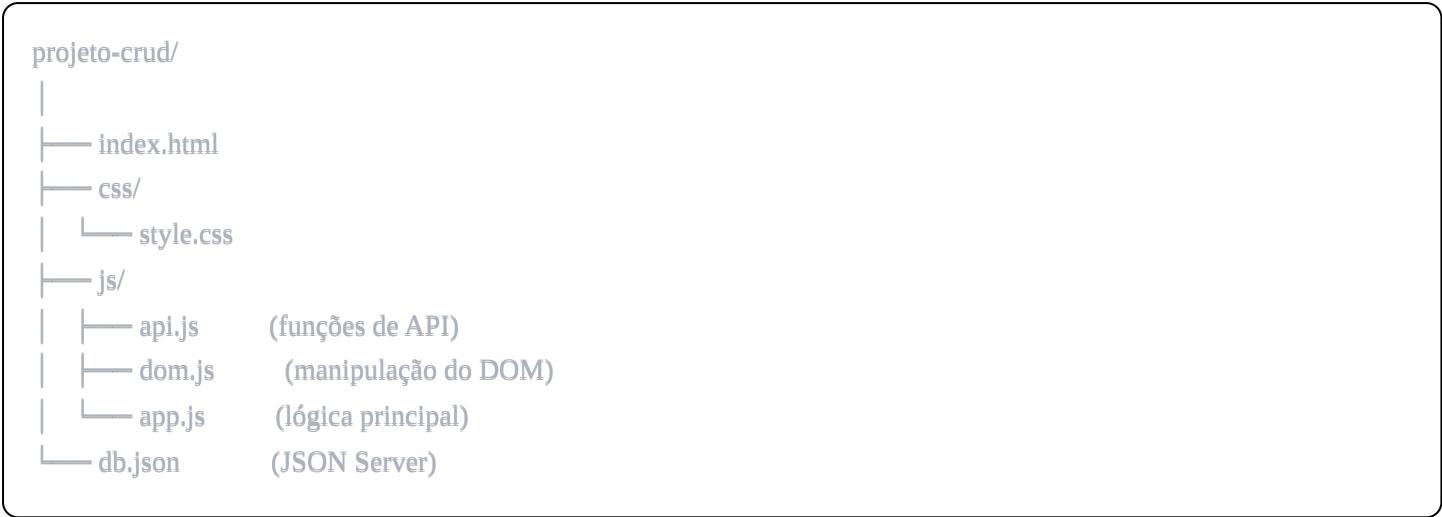
    alert('Produto atualizado com sucesso!');

  } catch (erro) {
    console.error('Erro ao editar produto:', erro);
    alert('Falha ao editar produto');
  }
}

// Carrega os produtos ao abrir a página
carregarProdutos();
```

9. Organização de Código para Aplicação CRUD Básica

Estrutura de Pastas



1. api.js - Camada de API

javascript

// api.js - Todas as chamadas à API centralizadas

```
const API_CONFIG = {  
  baseUrl: 'http://localhost:3000',  
  headers: {  
    'Content-Type': 'application/json'  
  }  
};
```

// Funções CRUD genéricas

```
const api = {  
  // GET - Buscar todos  
  async getAll(recurso) {  
    try {  
      const response = await fetch(`${API_CONFIG.baseUrl}/${recurso}`);  
      if (!response.ok) throw new Error('Erro ao buscar dados');  
      return await response.json();  
    } catch (erro) {  
      console.error(`Erro em getAll(${recurso}):`, erro);  
      throw erro;  
    }  
  },
```

// GET - Buscar por ID

```
  async getById(recurso, id) {  
    try {  
      const response = await fetch(`${API_CONFIG.baseUrl}/${recurso}/${id}`);  
      if (!response.ok) throw new Error('Erro ao buscar item');  
      return await response.json();  
    } catch (erro) {  
      console.error(`Erro em getById(${recurso}, ${id}):`, erro);  
      throw erro;  
    }  
  },
```

// POST - Criar

```
  async create(recurso, dados) {  
    try {  
      const response = await fetch(`${API_CONFIG.baseUrl}/${recurso}`, {  
        method: 'POST',  
        headers: API_CONFIG.headers,  
        body: JSON.stringify(dados)  
      });  
      if (!response.ok) throw new Error('Erro ao criar item');  
      return await response.json();  
    } catch (erro) {  
      console.error(`Erro em create(${recurso}):`, erro);
```

```
        throw erro;
    }
},

// PUT - Atualizar
async update(recurso, id, dados) {
    try {
        const response = await fetch(`${API_CONFIG.baseUrl}/${recurso}/${id}`, {
            method: 'PUT',
            headers: API_CONFIG.headers,
            body: JSON.stringify(dados)
        });
        if (!response.ok) throw new Error('Erro ao atualizar item');
        return await response.json();
    } catch (erro) {
        console.error(`Erro em update(${recurso}, ${id}):`, erro);
        throw erro;
    }
},

// DELETE - Deletar
async delete(recurso, id) {
    try {
        const response = await fetch(`${API_CONFIG.baseUrl}/${recurso}/${id}`, {
            method: 'DELETE'
        });
        if (!response.ok) throw new Error('Erro ao deletar item');
        return true;
    } catch (erro) {
        console.error(`Erro em delete(${recurso}, ${id}):`, erro);
        throw erro;
    }
}
};
```

2. dom.js - Manipulação do DOM

javascript

// dom.js - Funções para manipular a interface

```
const dom = {
```

// Selecionar elementos

```
getId(id) {
```

```
  return document.getElementById(id);
```

```
},
```

// Criar card de produto

```
criarCardProduto(produto) {
```

```
  const card = document.createElement('div');
```

```
  card.className = 'produto-card';
```

```
  card.id = `produto-${produto.id}`;
```

```
  card.innerHTML = `
```

```
    <h3>${produto.nome}</h3>
```

```
    <p><strong>Preço:</strong> R$ ${produto.preco.toFixed(2)}</p>
```

```
    <p><strong>Estoque:</strong> ${produto.estoque}</p>
```

```
    <button class="btn-editar" data-id="${produto.id}">Editar</button>
```

```
    <button class="btn-deletar" data-id="${produto.id}">Deletar</button>
```

```
  `;
```

```
  return card;
```

```
},
```

// Renderizar lista de produtos

```
renderizarProdutos(produtos, containerId) {
```

```
  const container = this.getId(containerId);
```

```
  container.innerHTML = '';
```

```
  produtos.forEach(produto => {
```

```
    const card = this.criarCardProduto(produto);
```

```
    container.appendChild(card);
```

```
  });
```

```
},
```

// Limpar formulário

```
limparFormulario(formId) {
```

```
  const form = this.getId(formId);
```

```
  if (form) form.reset();
```

```
},
```

// Mostrar mensagem

```
mostrarMensagem(mensagem, tipo = 'info') {
```

```
  const cores = {
```

```
    sucesso: '#4CAF50',
```

```
    erro: '#f44336',
```

```
    info: '#2196F3'
```

```
  };
```

```

const div = document.createElement('div');
div.style.cssText = `
  position: fixed;
  top: 20px;
  right: 20px;
  padding: 15px 25px;
  background-color: ${cores[tipo]};
  color: white;
  border-radius: 5px;
  box-shadow: 0 2px 5px rgba(0,0,0,0.2);
  z-index: 1000;
`;
div.textContent = mensagem;
document.body.appendChild(div);

setTimeout(() => div.remove(), 3000);
},

// Mostrar loading
mostrarLoading(containerId) {
  const container = this.getId(containerId);
  container.innerHTML = '<p>Carregando...</p>';
}
};

```

3. app.js - Lógica Principal

javascript

```

// app.js - Lógica de negócio e controle
const app = {
  recurso: 'produtos',

  // Inicializar aplicação
  async init() {
    await this.carregarProdutos();
    this.configurarEventos();
  },

  // Carregar produtos
  async carregarProdutos() {
    try {
      dom.mostrarLoading('listaProdutos');
      const produtos = await

```